

QUALITY SPRINT · WEEK ONE

# The Field Manual

---

Four missions to find out whether EchoBrief actually does what its landing page promises — and to leave it measurably better than you found it.

OWNER

Adnan

REVIEWER

Khush

WINDOW

21–28 Aug 2026

VERSION

v1.0

## READ THIS FIRST

# How to use this manual

This is not a reading document. It is a worksheet. Every mission ends in a file you commit to the repo — a scorecard, a report, or a pull request. If a day ends with no artefact, that day did not happen.

## What this week is

EchoBrief works end-to-end. A bot joins a call, the audio is transcribed, and a summary lands by email. That is *proven*. What is **not** proven is whether it is any good — across languages, across long meetings, and against the note-takers we are competing with.

Your job is to convert opinions into measurements.

## The one rule that matters

**Don't change code you're less than 95% sure about.** This is a live product with real users. A wrong fix is worse than an unfixed bug you documented clearly.

When under 95%: write down what you observed, what you think is happening, and what you'd change. Then ask. That's a win, not a failure.

## ANATOMY OF A MISSION PAGE

### QUESTION

The single question the mission answers. If your work doesn't answer it, you're off-track.

### HYPOTHESIS

What we already suspect is true. **Try to prove it wrong.** A hypothesis you confirm without trying to break it is worth nothing.

### METHOD

Exact steps and commands. Follow them; deviate only with a written reason.

### SCORECARD

The table you fill in. This is the deliverable.

### DONE WHEN

The bar. Not "I looked at it" — a specific, checkable state.

### BIAS WARNING — READ TWICE

You will be tempted to make EchoBrief look good. Resist it hard. The most valuable thing you can hand over on Friday is a list of things that are broken and a number attached to each one. **Finding nothing is the only failing outcome.** Every real product has failures at this stage; if your report has none, you tested too gently.

## Time budget

Friday afternoon for setup, then Mon–Fri for the work. Missions 1 & 2 are testing, 3 is judgement, 4 is product thinking. Roughly 1.5 / 1 / 1.5 / 0.5 days.

## Where work lives

Branch `intern/quality-sprint`. All artefacts go in `reports/2026-08-quality-sprint/`. Never commit to `main`.

## When stuck

Timebox to 45 minutes. Then post: what you tried, what you expected, what happened, the exact error. Don't sit silent for hours.

## PRODUCT MAP

# What you are testing

EchoBrief is an AI meeting note-taker built for Indian teams. A bot joins the call, records it, transcribes it, and emails a summary. The differentiator we sell is **Indian languages** — that claim is Mission 1.

## What we promise publicly

|                  |                                                              |
|------------------|--------------------------------------------------------------|
| <b>Languages</b> | "22 Indian languages"; with code-mixing (Hinglish, Tanglish) |
| <b>Delivery</b>  | Summary on email or WhatsApp, "per-language routing"         |
| <b>Auto-join</b> | Bot joins calendar meetings automatically                    |
| <b>Output</b>    | Summary, action items, decisions, risks, timeline, metrics   |

Sources: `src/components/landing/*`,  
`src/pages/Terms.tsx`

## What the code actually does

|                 |                                                                                                        |
|-----------------|--------------------------------------------------------------------------------------------------------|
| <b>STT</b>      | Sarvam <code>saaras:v3</code> , <code>mode: "translate"</code> , <code>language_code: "unknown"</code> |
| <b>Meaning</b>  | Auto-detects the language and outputs <b>English</b> , always                                          |
| <b>Fallback</b> | OpenAI Whisper if Sarvam fails or returns empty                                                        |
| <b>Insights</b> | GPT-4o-mini over the (English) transcript                                                              |

Source: `supabase/functions/_shared/sarvam.ts`

### THE GAP YOU SHOULD ALREADY BE ABLE TO SMELL

We advertise transcription *in 22 languages* and delivery *in the reader's language*. The pipeline translates everything to English and has no output-language switch anywhere. Whether that is a **bug**, a **missing feature**, or a **marketing-copy problem** is one of the things you're going to settle this week. Don't assume which.

### THE SURFACES YOU WILL TOUCH

#### WEB APP

##### React + Vite

Dashboard, meeting detail, metrics. `npm run dev` → :8080

#### BACKEND

##### Supabase

Postgres + 27 Deno edge functions

#### SPLITTER

##### Vercel fn

`api/split-audio.ts`,  
ffmpeg chunking

#### VENDORS

##### 3 APIs

Recall.ai · Sarvam · OpenAI

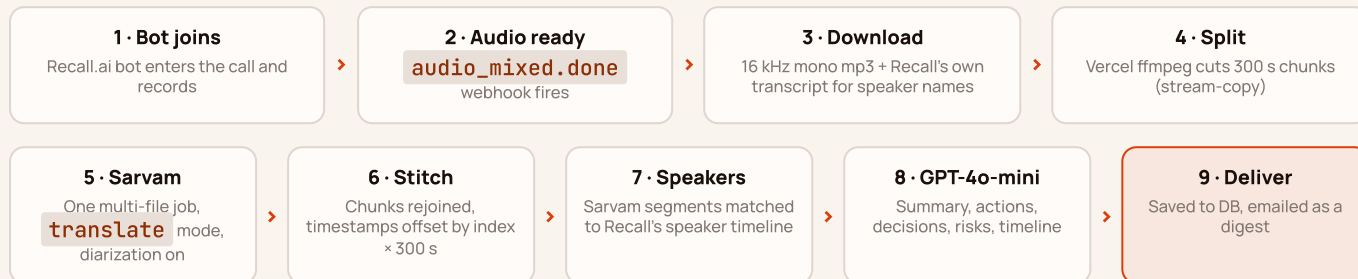
### Read these on Day 0, Friday evening — in this order

- |    |                                                                       |                                                                       |
|----|-----------------------------------------------------------------------|-----------------------------------------------------------------------|
| 01 | <code>docs/architecture.md</code>                                     | How the pieces fit and why. 15 min.                                   |
| 02 | <code>docs/pipeline.md</code>                                         | Every stage of processing, and the races it survives. 20 min.         |
| 03 | <code>errors.md</code>                                                | Every failure we've already seen. You will hit some of these. 10 min. |
| 04 | <code>docs/engineering-notes.md</code>                                | 23 write-ups of real problems. Skim; return when stuck.               |
| 05 | <code>docs/testing.md</code> +<br><code>scripts/evals/EVALS.md</code> | The harnesses you'll live in for Mission 3.                           |

## THE MACHINE

# How a meeting becomes a summary

Nine stages. Every mission this week is really a question about one or two of them. Learn where each mission bites and you'll debug ten times faster.



## WHERE EACH MISSION BITES

- M1** **Stage 5.** Does Sarvam handle each language? Does `translate` mode make the language promise true or false?
- M2** **Stages 4–6.** Chunking and stitching. Long audio is where this pipeline has historically broken.
- M3** **Stages 5, 7, 8.** Word accuracy, speaker attribution, and whether GPT's claims are actually in the transcript.
- M4** **Stage 8–9.** What we compute and show after the meeting.

## Two things that will confuse you on day one

**Why chunking exists.** Sarvam's `saaras:v3` silently returns an *empty* transcript for long audio — no error, just nothing. A 47-minute file fails; the same file cut into 5-minute pieces succeeds. So we cut everything over ~6 minutes into 300-second chunks. This is a vendor bug we work around, not a design choice.

**Why there are two transcripts.** Recall produces its own rough transcript. We don't show it — we use it purely to learn *who spoke when*, because Sarvam gives good words but anonymous speakers. Stage 7 is the marriage of the two, and it is the least reliable stage in the system.

## FAILURE IS DESIGNED-IN, AND THAT MATTERS FOR YOUR TESTING

If Sarvam returns nothing, an error, or the known `KeyError: 'timestamps'` bug, the system **silently falls back to Whisper** and carries on. So "we got a transcript" does not mean "Sarvam worked". For every test you run, record *which engine produced the result* — otherwise Mission 1's language findings are meaningless.

## NAVIGATION

# Where things live

## FILES YOU'LL OPEN THIS WEEK

|                                               |                                            |
|-----------------------------------------------|--------------------------------------------|
| <code>_shared/sarvam.ts</code>                | STT job config — <b>M1</b>                 |
| <code>api/split-audio.ts</code>               | ffmpeg chunking — <b>M2</b>                |
| <code>_shared/stitch.ts</code>                | Chunk rejoin + offsets — <b>M2</b>         |
| <code>sarvam-webhook/index.ts</code>          | Speaker mapping, fallbacks — <b>M2, M3</b> |
| <code>_shared/insights.ts</code>              | The GPT prompt — <b>M3</b>                 |
| <code>_shared/metrics.ts</code>               | Computed conversation metrics — <b>M4</b>  |
| <code>meeting/MeetingMetrics.tsx</code>       | The metric tiles users see — <b>M4</b>     |
| <code>scripts/sarvam_config_test.py</code>    | Fire a custom Sarvam job — <b>M1</b>       |
| <code>scripts/evals/run_evals.py</code>       | Output-quality evals — <b>M3</b>           |
| <code>scripts/pipeline-test/harness.py</code> | 11-scenario prod harness — <b>all</b>      |

## VOCABULARY

|                        |                                                                                                    |
|------------------------|----------------------------------------------------------------------------------------------------|
| <b>Diarization</b>     | Splitting audio by <i>who is speaking</i> , producing SPEAKER_00, SPEAKER_01...                    |
| <b>Attribution</b>     | Turning those anonymous labels into real names                                                     |
| <b>Translate mode</b>  | Sarvam setting: speech in any Indic language → <b>English text</b>                                 |
| <b>Transcribe mode</b> | The alternative: output stays in the spoken language                                               |
| <b>WER</b>             | Word Error Rate. Errors + reference words. Lower is better; under 15% is usable, under 10% is good |
| <b>Code-mixing</b>     | Two languages in one sentence — Hinglish. Very common in real Indian meetings                      |
| <b>Hallucination</b>   | The model asserting something the transcript never said                                            |
| <b>Edge function</b>   | A Deno serverless function on Supabase                                                             |
| <b>Harness</b>         | Does the plumbing work? (pass/fail)                                                                |
| <b>Eval</b>            | Is the output any <i>good</i> ? (scored)                                                           |

## HARNESS VS EVAL — YOU WILL NEED THIS DISTINCTION ALL WEEK

The **harness** asks "did the machine run?". The **evals** ask "was what came out correct?". A meeting can complete with status `completed`, a green harness, and a summary that is total nonsense. Missions 1–3 live almost entirely on the eval side of that line.

## THE FOUR COMMANDS YOU'LL RUN CONSTANTLY

```
# 1. Unit tests — mocked, under a second, zero cost. Run after every code change.
npm run test:unit

# 2. Does the plumbing work end-to-end against real prod? ~90 s.
python3 scripts/pipeline-test/harness.py

# 3. Is the output any good? 8 scored evals, LLM judge. Run before touching insights.
python3 scripts/evals/run_evals.py

# 4. Fire one Sarvam job with whatever settings you want. Your M1 workhorse.
python3 scripts/sarvam_config_test.py <meeting_id> mode=transcribe language_code=hi-IN
```

DAY 0 · FRIDAY 21 AUG, FROM 16:00

# Get running before you test anything

You start **Friday 21 August at 16:00**. That afternoon is Day 0, and it is deliberately not testing — it is getting green, reading, and setting the slow things running. Target: everything below green by **19:00 Friday**. If you're not there, say so that evening rather than losing Monday to it. **The weekend is yours** — buffer, not schedule.

## Environment

- ☐ Repo cloned, on branch `intern/quality-sprint`
- ☐ `npm install` clean, no peer-dep errors
- ☐ Node 20+, Python 3.11+, Deno, Supabase CLI, `ffmpeg` on PATH
- ☐ `.env` received from Khush **over a secure channel** — never Slack/WhatsApp plaintext
- ☐ `supabase/.env.local` populated
- ☐ Supabase dashboard access confirmed (read)

## Smoke tests — all must pass

- ☐ `npm run dev` → app renders at :8080, you can log in
- ☐ `npm run build` → succeeds (brand check included)
- ☐ `npx tsc --noEmit` → note the baseline error count, it is not zero
- ☐ `npm run test:unit` → 53 tests pass
- ☐ `python3 scripts/pipeline-test/harness.py` → 11/11
- ☐ `python3 scripts/evals/run_evals.py` → passes, including the case designed to fail

### TRAP: `NPM RUN BUILD` DOES NOT TYPECHECK

Vite uses esbuild, which strips TypeScript types without checking them. A green build tells you nothing about type errors. Use `npx tsc --noEmit` separately, and compare against the baseline you recorded on Day 0 — some errors pre-date you.

## THEN START THESE — DAY 0 EVENING, FINISH MONDAY FIRST THING

### 1 Create the fixtures folder

`recordings/lang-fixtures/`, one file per language, named `<code>-<source>-<seconds>s.mp3` — e.g. `hi-IN-news-90s.mp3`. This naming is not decoration; your scorecard joins on it.

### 2 Normalise every fixture to the pipeline's native format

Recall hands us 16 kHz mono 128 kbps mp3. Match it exactly, or you are testing your own encoder rather than our pipeline: `ffmpeg -i in.wav -ar 16000 -ac 1 -b:a 128k out.mp3`

### 3 Write a reference transcript for each fixture

A plain `.txt` of what is actually said, plus an English gloss. Without a reference you cannot compute WER and Mission 3 collapses into vibes. Get a native speaker to verify anything you don't speak yourself — **do not trust an LLM to produce ground truth**.

### 4 Extend the config-test script to accept a local file

Today `sarvam_config_test.py` pulls audio from a stored meeting. Add a `--file` flag so it can upload a local fixture instead. Small change, unblocks the whole week. **This is your first PR**.

## WHERE TO GET AUDIO, IN ORDER OF PREFERENCE

1. Record yourself and a friend having a real 2-minute conversation — most realistic, and you get perfect ground truth for free. 2. Public-domain news bulletins and government broadcasts (All India Radio has all the scheduled languages). 3. Creative-Commons YouTube, downloaded audio-only. Log the source URL for every fixture. **Never use a customer's real meeting audio as a fixture.**

## SAFETY

# Red lines & the deploy gate

You have production access. That is trust, and it is reversible. These rules exist because every single one of them has already cost us a real outage.

## NEVER — NO EXCEPTIONS, NO "JUST THIS ONCE"

- Commit or push to `main`. Everything goes through a PR on `intern/quality-sprint`.
- Deploy an edge function on a **Friday**, or after 18:00 on any day.
- Run `DELETE` or `UPDATE` against a production table by hand.
- Raise any `pg_cron` frequency. The database is also our scheduler; frequent ticks drained the Disk IO Budget once already and took the whole pipeline down.
- Commit `.env`, an API key, a JWT, or a real customer transcript. Check `git diff --staged` every single time.
- Open, screenshot, or paste a real customer's meeting content anywhere. Use `[harness]` test meetings and your own fixtures.
- Change `src/components/ui/*` — those are generated shadcn files.
- Add a hex colour that isn't in `brand/tokens/colors.json`. The brand checker blocks the commit and it is right to.

## Free to do, no permission needed

- Read anything — code, docs, dashboards, logs
- Run any read-only SQL `SELECT`
- Run the harness, evals and unit tests as often as you like
- Create `[harness]`-prefixed test meetings (emails are suppressed for them)
- Fire Sarvam jobs on your own fixtures
- Open draft PRs with anything, however rough

## Ask first

- Any prod edge-function deploy
- Any database migration
- Starting a Recall bot on a real meeting (costs bot minutes)
- A test run you expect to cost more than ~₹500 in API credits
- Changing landing-page copy — that's a claim about the product, not a text edit
- Setting `HARNESS_EMAILS=true` (and unset it immediately after)

## DEPLOY GATE — EVERY BOX TICKED, IN ORDER, BEFORE ANY EDGE FUNCTION SHIPS

- ☐ `npm run test:unit` passes
- ☐ `npx tsc --noEmit` shows no *new* errors vs your Day 0 baseline
- ☐ `python3 scripts/pipeline-test/harness.py` → 11/11
- ☐ If you touched transcription or insights: `python3 scripts/evals/run_evals.py` passes
- ☐ If you touched the chunking or Sarvam path: harness re-run with `--live`
- ☐ PR opened, reviewed by Khush, approved in writing
- ☐ Deployed before 18:00, Monday–Thursday
- ☐ Harness re-run **after** deploy — a green pre-deploy run proves nothing about the deployed code
- ☐ You watched `monitor_events` and the function logs for 30 minutes afterwards

## IF YOU BREAK PRODUCTION

Say so immediately — in the channel, not in a DM, not after you've tried to fix it quietly. State what you changed, when, and what you're seeing. Then revert first and diagnose second. **Nobody has ever been in trouble here for breaking something and saying so within two minutes.** People get in trouble for the silent hour.

## THE PLAN

## Four missions, Friday to Friday

## MISSION 1 MON – TUE AM

## Does it really do 22 languages?

We sell "22 Indian languages". Prove or disprove it, language by language, with a number per language – not a yes/no.

## MISSION 2 TUE PM – WED AM

## Does it survive long meetings?

Real meetings are 30–60 minutes. Find where duration starts degrading quality, and where it breaks outright.

## MISSION 3 WED PM – THU

## Is the summary actually right?

Words, speakers, action items, decisions. Measure each separately – they fail for different reasons.

## MISSION 4 DAY 0 PREP + FRI AM

## Are our metrics worth showing?

Run one meeting through Fireflies, Otter and Granola. Compare. Kill what's noise, add what's missing, ship one change.

## TIMELINE

|                         | FRI 21<br>16:00→ | MON 24 | TUE 25 | WED 26 | THU 27 | FRI 28<br>→16:00 |
|-------------------------|------------------|--------|--------|--------|--------|------------------|
| Day 0 · setup & reading |                  |        |        |        |        |                  |
| M1 · Languages          |                  |        |        |        |        |                  |
| M2 · Long meetings      |                  |        |        |        |        |                  |
| M3 · Accuracy           |                  |        |        |        |        |                  |
| M4 · Metrics            |                  |        |        |        |        |                  |
| Report & readout        |                  |        |        |        |        |                  |

## DAILY RHYTHM — NON-NEGOTIABLE

|        |                            |                                                                                                                                                           |
|--------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| FRI 21 | Day 0, 16:00–19:00         | Setup green, docs read, and the three competitor accounts created with one meeting already uploaded to each – those results sit waiting for Friday 28.    |
| 09:30  | Written standup            | Three lines in the channel: what you finished, what's next, what's blocking. Written, not verbal – it becomes the log.                                    |
| 09:45  | Execute                    | Head down. Follow the mission page. Timebox any blocker to 45 minutes before escalating.                                                                  |
| 17:00  | Commit the day's artefacts | Scorecard rows, notes, screenshots – pushed. An uncommitted finding is a lost finding.                                                                    |
| 17:30  | 15-min sync with Khush     | Show the numbers you got today, not a description of what you did.                                                                                        |
| FRI 28 | Different shape            | Mission 4 in the morning, everything committed by 15:00, readout at 16:00. No new testing after lunch – Friday afternoon is for writing, not discovering. |

## IF YOU FALL BEHIND — CUT IN THIS ORDER

The shipped change in M4 first (leave it as a written proposal), then the long tail of M1's rarer languages (do the top 8 properly rather than 22 badly), then M2's extra duration steps. **Never cut Mission 3.** Accuracy is the whole product; everything else is packaging.

# 01 Does it really work in 22 languages?

Stage 5 of the pipeline · Sarvam saaras:v3

MON - TUE AM

## THE QUESTION

**If a customer holds a meeting in Marathi, or Kannada, or Hinglish — do they get a summary they'd actually accept? And is our public claim of "22 Indian languages" honest as written?**

## HYPOTHESES — YOUR JOB IS TO ATTACK THESE, NOT CONFIRM THEM

- H1** Sarvam *documents* all 22 scheduled languages + English on `saaras:v3`. Documented support and usable quality are different things. Expect the top 8 by speaker count to be good and the long tail (Santali, Bodo, Dogri, Manipuri, Kashmiri, Sanskrit) to be poor or empty.
- H2** Because we run `mode: "translate"`, the user **always** gets English — never a transcript in their own language. Our copy implies otherwise.
- H3** "Per-language routing" is advertised in the FAQ and pricing. `preferred_languages` is collected at onboarding and then **read by nothing**. Verify this yourself: `grep -rn "preferred_languages" src/ supabase/`
- H4** Auto-detect ( `language_code: "unknown"` ) is weakest on code-mixed audio, where Hinglish may be detected as English and the Hindi half quietly dropped.
- H5** The landing-page list shows 21 Indian languages plus English, omitting Nepali — so "22 Indian languages" is arguably miscounted. Confirm by reading `src/components/Landing/Languages.tsx`.

## METHOD

### 1 Build the fixture set — 24 clips

One 60–120 s clip per language on the landing page, **plus two code-mixed clips** (Hinglish, Tanglish) since that's what real Indian meetings sound like. Each needs a human-verified reference transcript.

### 2 Run every fixture three ways

(a) production settings — `translate` + `unknown`; (b) `translate` with the language pinned; © `transcribe` with the language pinned, i.e. output in the native script. Run © even though it isn't what prod does — it separates "Sarovam can't hear this language" from "our config is wrong", and those have completely different fixes.

### 3 Record the detected language and confidence

Sarovam returns a `language_probability` when detecting. Log it. A confident wrong detection is a much more serious finding than a low-confidence one.

### 4 Score each result

Use the 0–4 rubric overleaf. Where you have a reference translation, also compute WER. Get a native speaker to sanity-check any language you don't speak — one 10-minute call per language is cheap and it is the only thing that makes the score credible.

### 5 Push the best and the worst language through the whole pipeline

Transcript quality isn't the product — the *summary* is. Take your highest and lowest scoring languages, run them through to insights, and check whether a mediocre transcript still yields a usable summary. Sometimes it does. That finding changes what we fix first.

## MISSION 1 · DELIVERABLE

## The language scorecard

Commit as `reports/2026-08-quality-sprint/m1-languages.csv` — one row per language.

| LANGUAGE              | CODE    | DETECTED | CONF. | SCORE 0-4 | WER | FAILURE NOTE |
|-----------------------|---------|----------|-------|-----------|-----|--------------|
| Hindi                 | hi-IN   |          |       |           |     |              |
| English (Indian)      | en-IN   |          |       |           |     |              |
| Bengali               | bn-IN   |          |       |           |     |              |
| Marathi               | mr-IN   |          |       |           |     |              |
| Telugu                | te-IN   |          |       |           |     |              |
| Tamil                 | ta-IN   |          |       |           |     |              |
| Gujarati              | gu-IN   |          |       |           |     |              |
| Urdu                  | ur-IN   |          |       |           |     |              |
| Kannada               | kn-IN   |          |       |           |     |              |
| Odia                  | od-IN   |          |       |           |     |              |
| Malayalam             | ml-IN   |          |       |           |     |              |
| Punjabi               | pa-IN   |          |       |           |     |              |
| Assamese              | as-IN   |          |       |           |     |              |
| Maithili              | mai-IN  |          |       |           |     |              |
| Sanskrit              | sa-IN   |          |       |           |     |              |
| Konkani               | kok-IN  |          |       |           |     |              |
| Dogri                 | doi-IN  |          |       |           |     |              |
| Sindhi                | sd-IN   |          |       |           |     |              |
| Manipuri              | mni-IN  |          |       |           |     |              |
| Bodo                  | brx-IN  |          |       |           |     |              |
| Santali               | sat-IN  |          |       |           |     |              |
| Kashmiri              | ks-IN   |          |       |           |     |              |
| Hinglish (mixed)      | unknown |          |       |           |     |              |
| Tanglish (mixed)      | unknown |          |       |           |     |              |
| Nepali — on our list? | ne-IN   |          |       |           |     |              |

## Scoring rubric — be strict

- 0 Empty, error, or unusable garbage
- 1 Recognisable topic, most content wrong. Not shippable.
- 2 Broadly right, frequent errors. A user would complain.
- 3 Good. Occasional name or number wrong. A user would accept it.
- 4 Near-perfect, including proper nouns and figures.

**3 is the shipping bar.** A language scoring 2 or below cannot honestly appear on our website.

## ANSWER THESE IN THE WRITE-UP

- How many languages score  $\geq 3$ ? Is that number 22?
- Does auto-detect ever pick the wrong language *with high confidence*?
- What happens to Hinglish specifically — is the Hindi dropped?
- Does **transcribe** mode work where **translate** fails, or vice versa?
- Which failed silently vs. loudly? Silent is worse.
- Exact wording** you'd put on the site instead, if the claim doesn't hold.

## DONE WHEN

All 25 rows filled with a score and evidence · a one-page verdict stating how many languages we can honestly claim · a written recommendation on the marketing copy · and **H2 and H3 answered with a file-and-line citation**, not an opinion.

## 02 Does it survive a 45-minute meeting?

Stages 4–6 · split-audio → Sarvam → stitch

TUE PM – WED AM

### THE QUESTION

**Real meetings are 30–60 minutes. At what duration does quality start to degrade, and at what duration does it break outright?**

### BACKGROUND YOU NEED

Sarvam's `saaras:v3` silently returns an **empty** transcript on long audio — a confirmed vendor bug. A 47-minute file returned nothing; the same file in 5–6 minute chunks worked. So anything over ~6 minutes is cut into **300-second chunks** by a Vercel ffmpeg function, submitted as one multi-file job, then stitched back together with each chunk's timestamps offset by `index × 300`.

That machinery is where long meetings break. A 45-minute meeting is **9 chunks**; one bad chunk is 11% of the meeting silently missing, and nothing in the UI will tell the user.

### Four failure modes to hunt for

|                           |                                                                                                                              |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Empty chunk</b>        | One chunk returns nothing; the stitch succeeds anyway and 5 minutes vanish                                                   |
| <b>Offset drift</b>       | Timestamps after chunk 1 are wrong, so the timeline and speaker mapping slide out of sync                                    |
| <b>Boundary loss</b>      | A sentence spanning a cut is mangled or duplicated at both edges                                                             |
| <b>Summary flattening</b> | Transcript is fine, but GPT compresses 45 minutes into the same 5 bullets it gives a 10-minute call — early topics disappear |

### METHOD – THE DURATION LADDER

#### 1 Build one long fixture, then cut it down

Take a single ~90-minute clear multi-speaker recording (a panel discussion, a podcast, a recorded internal call) and produce truncations at **5, 15, 30, 45, 60, 90 minutes**. Same content, same voices — so duration is the *only* variable. Comparing six different recordings would tell you nothing.

#### 2 Push each through the real production path

Splitter → Sarvam → webhook → stitch → insights. Not the local mock. If the splitter is bypassed you're not testing the thing that breaks.

#### 3 Measure five things per run

Chunk count · **non-empty chunks / total chunks** · total transcript characters · whether timestamps increase monotonically across every boundary · wall-clock from submission to `completed`.

#### 4 Check for silent content loss

Characters should scale roughly linearly with duration. If 30 min gives ~28,000 characters and 45 min gives ~30,000, you have found missing content — **even though the meeting shows as completed**. This is the single most valuable bug you can find this week.

#### 5 Read the summaries side by side

For 15, 45 and 90 minutes: does the summary of the 90-minute run actually mention what happened in the first 15 minutes? Count action items found at each duration. Watch for the count going *flat* as the meeting gets longer — that's the flattening failure.

### ALSO CHECK THE EDGES, NOT JUST THE MIDDLE

Two known sharp corners. **Under 6 minutes**, chunking is skipped entirely — a different code path, so test 5 minutes deliberately. **Whisper OOMs** in the edge function above roughly 15 MB of audio, so if a long meeting falls back from Sarvam to Whisper it may die a second time. Force that path once and record what the user would see.

## MISSION 2 · DELIVERABLE

# The duration ladder

Commit as `reports/2026-08-quality-sprint/m2-duration.csv`.

DURATION   CHUNKS   NON-EMPTY   CHARS   CHARS/MIN   TIME TO DONE   TIMESTAMPS   OK   OUTCOME

5 min

NO CHUNKING

15 min

30 min

45 min

60 min

90 min

**Chars/min is the column that matters.** It should stay roughly flat down the table. Any drop is content being lost silently. Reference point from a known-good run: a 29-minute Recall recording produced ~28,400 characters across 6 non-empty chunks.

## SUMMARY QUALITY BY DURATION

DURATION   ACTION   ITEMS   COVERS   FIRST 15 MIN?   SCORE 0-4

15 min

45 min

90 min

If action-item count doesn't grow with duration, the summary is being flattened. That's a prompt problem, not a transcription problem — and it's fixable in a day.

## ANSWER IN THE WRITE-UP

- What is the longest duration that works **reliably**?
- Where exactly does it start degrading — and does the user get any warning?
- Did any chunk come back empty? Which index, and could you reproduce it?
- Do timestamps stay monotonic across every boundary?
- How long does a 45-minute meeting take end to end? Would you wait that long?
- Should we **surface** partial-transcript failures to the user instead of hiding them?

## USEFUL PROBES

```
# Is a chunked job actually returning content for every chunk?
python3 scripts/check_sarvam_job.py <job_id>

# Full pipeline against real prod, including the live Sarvam contract check (~3 min extra)
python3 scripts/pipeline-test/harness.py --live

# Chunk-stitch offset maths, mocked and instant
npm run test:unit
```

## DONE WHEN

Six duration rows complete · a stated maximum reliable duration with evidence · every failure reproduced at least twice (once is an anecdote, twice is a bug) · and each finding filed either as a new entry in `errors.md` or as a GitHub issue with reproduction steps.

## 03 Is the summary actually correct?

Stages 5, 7 & 8 · words, speakers, insights

WED PM - THU

### THE QUESTION

**If a user reads only our summary and never the transcript, will they be correctly informed — or confidently misled?**

### MEASURE FOUR THINGS SEPARATELY — THEY FAIL FOR DIFFERENT REASONS AND HAVE DIFFERENT FIXES

- |                                |                                                                                              |
|--------------------------------|----------------------------------------------------------------------------------------------|
| <b>1 · Word accuracy</b>       | Did it hear the words? → fix = Sarvam config or audio quality                                |
| <b>2 · Speaker attribution</b> | Did it get who said what? → fix = the Recall-timeline mapping in <code>sarvam-webhook</code> |
| <b>3 · Extraction</b>          | Did it find the real action items and decisions? → fix = the GPT prompt                      |
| <b>4 · Faithfulness</b>        | Did it invent anything? → fix = prompt constraints + hallucination checks                    |
- A single "accuracy: 78%" number is useless to whoever has to fix it. Four numbers are actionable.

### METHOD

#### 1 Build a golden set of 5 meetings

Aim for: one clean English 2-person call · one Hinglish 3-person call · one noisy call (background sound, a phone speaker) · one long call from Mission 2 · one meeting where people **talk over each other**. That last one is where every note-taker on the market falls apart, so it's where you find something worth reporting.

#### 2 Write ground truth by hand

For each: the true transcript, the true speaker labels, and a human list of every action item and decision. This is slow — budget three hours — and it is the entire foundation of the mission. Don't let an LLM write it; you'd be grading the model against itself.

#### 3 Compute WER per meeting

Normalise both sides first — lowercase, strip punctuation — then use `jiwer`. Report overall WER **and** a separate error count for names, numbers and dates, because those are the words that actually matter in a meeting summary.

#### 4 Score speaker attribution

Percentage of segments assigned to the right person. Note separately whether any `SPEAKER_00`-style label leaked through to the UI — that's a visible product bug, not just an accuracy number.

#### 5 Score action items with precision and recall

**Recall** = of the real action items, how many did we find? **Precision** = of the ones we listed, how many were real? Low recall means users miss things. Low precision means users stop trusting the tool. State clearly which one we have.

#### 6 Hunt hallucinations deliberately

For every claim in a summary, find the supporting line in the transcript. Pay special attention to **owners** ("Rahul will send the deck" — did Rahul actually say that, or did the model assign it?) and to `sentiment_score`, which is a pure model guess with no measurement behind it.

#### 7 Fold your golden set into the eval harness

`python3 scripts/evals/run_evals.py --snapshot <meeting-id>` pulls a real meeting into the eval dataset. Do this for every golden meeting. **This is the highest-leverage hour of your week** — it turns your one-off testing into a permanent regression gate.

MISSION 3 · DELIVERABLE

# The accuracy scorecard

Commit as `reports/2026-08-quality-sprint/m3-accuracy.csv` , plus the golden set under `reports/2026-08-quality-sprint/golden/` .

| GOLDEN MEETING              | WER | NAME/NUMBER | ERRORS | SPEAKER | ACC. | AI | RECALL | AI | PRECISION | HALLUCINATIONS | VERDICT |
|-----------------------------|-----|-------------|--------|---------|------|----|--------|----|-----------|----------------|---------|
| 1 · Clean English, 2 people |     |             |        |         |      |    |        |    |           |                |         |
| 2 · Hinglish, 3 people      |     |             |        |         |      |    |        |    |           |                |         |
| 3 · Noisy / phone speaker   |     |             |        |         |      |    |        |    |           |                |         |
| 4 · Long (45 min+)          |     |             |        |         |      |    |        |    |           |                |         |
| 5 · Overlapping speech      |     |             |        |         |      |    |        |    |           |                |         |
| Average                     |     |             |        |         |      |    |        |    |           |                |         |

### Interpreting the numbers

|                   |               |                                                                                    |
|-------------------|---------------|------------------------------------------------------------------------------------|
| WER < 10%         | <b>GOOD</b>   | competitive with Otter on English                                                  |
| 10–20%            | <b>USABLE</b> | summary survives; verbatim quotes don't                                            |
| > 20%             | <b>BROKEN</b> | summaries will be wrong, not just rough                                            |
| Speaker > 90%     |               | Acceptable                                                                         |
| Speaker < 75%     |               | Worse than useless — confidently wrong attribution                                 |
| Any hallucination |               | Report it individually. One invented decision destroys trust in the whole product. |

### ANSWER IN THE WRITE-UP

- Which of the four dimensions is weakest? **Fix order follows this answer.**
- Are we losing more on *hearing* or on *reasoning*?
- Do names and numbers fail worse than ordinary words?
- Does overlapping speech break attribution, the words, or both?
- Did any `SPEAKER_XX` label reach the UI or the email?
- Is `sentiment_score` ever meaningfully right — or should it be removed?

### ONE PROMPT EXPERIMENT, IF TIME ALLOWS

The GPT prompt lives in `_shared/insights.ts` . If Mission 3 shows an extraction problem rather than a transcription problem, try **one** tightly-scoped prompt change — for example, requiring each action item to quote the transcript line it came from. Then re-run `run_evals.py` and report the before/after scores. **Change one thing at a time and measure it.** A prompt change with no eval number attached cannot be reviewed and won't be merged.

### DONE WHEN

Five golden meetings built with human ground truth · all six columns filled · every hallucination logged verbatim with the transcript line that disproves it · golden meetings snapshotted into `scripts/evals/dataset/` · and a ranked list of the top three accuracy fixes, most valuable first.

## 04 Are our post-meeting metrics worth showing?

Stages 8–9 · what the user sees afterwards

DAY 0 PREP · FRI AM

### THE QUESTION

A user finishes a meeting and opens EchoBrief. Which numbers change what they do next — and which are just decoration we should delete?

### WHAT WE SHOW TODAY — AUDIT EVERY ROW

| TILE            | WHAT IT IS                            | SOURCE    | YOUR VERDICT: KEEP / KILL / CHANGE |
|-----------------|---------------------------------------|-----------|------------------------------------|
| Talk time       | Seconds and % per speaker             | COMPUTED  |                                    |
| Speaking rate   | Words per minute                      | COMPUTED  |                                    |
| Silence         | % of meeting with no speech           | COMPUTED  |                                    |
| Speaker turns   | Number of hand-offs                   | COMPUTED  |                                    |
| Longest stretch | Longest monologue (15 s gap ends one) | COMPUTED  |                                    |
| Dead air        | Silence before the first word         | COMPUTED  |                                    |
| Balance         | 1 – Gini over speaking seconds        | COMPUTED  |                                    |
| Sentiment       | Positive / Neutral / Negative         | GPT GUESS |                                    |

Seven of the eight are computed deterministically from segment timings in `_shared/metrics.ts` — they are real measurements and unit-tested. **Sentiment is the odd one out:** a single number a language model volunteers, with nothing measuring it. Treat that asymmetry as a finding, not a footnote.

### THE UNCOMFORTABLE QUESTION TO ACTUALLY ANSWER

"Balance: 0.72" — what does a user *do* differently having seen that? If you can't answer for a metric, that metric is decoration. Great analytics change a behaviour; the rest is a dashboard nobody reopens. Judge all eight tiles against that single test.

### METHOD — THE SAME-MEETING TEST

- Sign up for the free tiers — do this on Day 0, Friday 21**  
Fireflies.ai, Otter.ai, Granola. Use a work email, not a customer's. Accounts need verifying and their processing takes time, so start them a week early and this mission becomes analysis rather than waiting. Note what each free tier limits — that's competitive intel in itself.
- Run *one identical meeting* through all four — also Day 0**  
Same audio, ours included. This is the only way to compare fairly. Anything else is comparing their best demo against our average case.
- Screenshot every post-meeting screen**  
Store under `reports/2026-08-quality-sprint/competitors/`. Redact anything identifying. Screenshots make the argument; prose alone won't.
- Fill the comparison grid overleaf, then decide**  
Three to kill, three to add, ranked by value + effort. Then **implement exactly one** and open a PR. One shipped change beats a ten-item wishlist.

MISSION 4 · DELIVERABLE

Us versus them

Mark each: YES PARTIAL NO . Add any row you find that isn't listed.

| CAPABILITY                               | ECHOBRIEF | FIREFLIES | OTTER | GRANOLA | NOTES |
|------------------------------------------|-----------|-----------|-------|---------|-------|
| Talk-time split per speaker              |           |           |       |         |       |
| Talk-to-listen ratio                     |           |           |       |         |       |
| Question count / rate                    |           |           |       |         |       |
| Monologue & longest-stretch alert        |           |           |       |         |       |
| Filler-word count                        |           |           |       |         |       |
| Sentiment                                |           |           |       |         |       |
| Topic / chapter breakdown                |           |           |       |         |       |
| Action items with owners                 |           |           |       |         |       |
| Action items carried across meetings     |           |           |       |         |       |
| Trends over time / per person            |           |           |       |         |       |
| Ask-questions-of-the-transcript chat     |           |           |       |         |       |
| Clip / share a moment                    |           |           |       |         |       |
| Jump from summary line to that timestamp |           |           |       |         |       |
| Meeting-cost estimate (people × time)    |           |           |       |         |       |
| Indian-language support                  |           |           |       |         |       |
| Time to summary after the call ends      |           |           |       |         |       |

**Kill – 3 metrics**

Not measured, not actionable, or nobody looks at it.

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

**Add – 3 metrics**

Competitors have them, or our data already supports them for free.

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

**Ship – 1 change**

Highest value + effort. PR by Friday noon.

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

CHEAP WINS ALREADY SITTING IN OUR DATA

Every segment carries exact start/end times, and `metrics.ts` already counts **questions** and **words per speaker** – both computed, both currently **never displayed**. Read that file before proposing anything new: the highest-value change this week may be surfacing a number we already calculate, which is a frontend-only PR with zero pipeline risk.

DONE WHEN

Grid complete with screenshots for all four products · kill/add lists written with a one-line reason each · one change implemented, tested and opened as a PR · and a short written argument for **what makes EchoBrief different**, given we will never out-feature Otter on English.

## FRIDAY

# What you hand over

One branch, one PR, one folder. If it isn't in the repo by **Friday 15:00** — an hour before the readout — it doesn't exist.

```
reports/2026-08-quality-sprint/
├── REPORT.md           – the main deliverable, structure below
├── m1-languages.csv    25 rows, scored
├── m2-duration.csv     6 rows + summary-quality table
├── m3-accuracy.csv     5 golden meetings, 6 metrics each
├── m4-competitors.md   grid + kill/add lists
├── golden/            ground-truth transcripts & action-item lists
├── competitors/       screenshots, redacted
└── raw/              job ids, logs, anything you'd need to re-check a number
```

## REPORT.MD — USE EXACTLY THIS STRUCTURE

- Verdict** — 5 bullets. If Khush reads only this, what must they know?
- Findings** — one block each: what you did, the number, the evidence, what it means
- Bugs found** — severity, reproduction steps, suspected cause, linked issue
- Fix backlog** — ranked by value + effort, with your estimate for each
- Claims to correct** — anything the website says that the code doesn't do
- What I couldn't test and why** — this section is *mandatory* and must not be empty

Aim for four pages. Nobody reads twelve. Every claim gets a number or a file-and-line citation.

## How to write a finding

### Weak

"Kannada didn't work very well, the transcript looked bad."

### Strong

"Kannada scored 1/4 (WER 41%, fixture **kn-IN-news-90s.mp3**, Sarvam job **abc-123**). Auto-detect returned **te-IN** at 0.81 confidence — confidently wrong, not uncertain. Pinning **language\_code=kn-IN** raised the score to 3/4 and dropped WER to 12%. **Implication:** auto-detect is the failure, not the model. If we let users declare their meeting language, Kannada becomes shippable."

The second one is worth a hundred times the first, because someone can **act** on it this afternoon.

## DEFINITION OF DONE — THE GRADING RUBRIC, SHOWN TO YOU UP FRONT ON PURPOSE

| DIMENSION       | MEETS THE BAR                                          | EXCEEDS IT                                                            |
|-----------------|--------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Coverage</b> | All four missions attempted; scorecards filled         | Edge cases tested that this manual didn't ask for                     |
| <b>Rigour</b>   | Every claim has a number and a reproducible source     | Failures reproduced twice and root-caused to a file and line          |
| <b>Honesty</b>  | Failures reported plainly, including your own mistakes | You actively tried to disprove your own conclusions                   |
| <b>Leverage</b> | Findings written up so someone else can act on them    | Golden set added to the evals, so this never regresses silently again |
| <b>Shipping</b> | At least one merged PR                                 | A merged PR with an eval number proving it improved something         |

## FRIDAY 16:00 · 30-MINUTE READOUT

Walk through the verdict and the top three findings. Bring the numbers, not the narrative. Expect to be asked "*how do you know?*" about every single claim — and be ready to say "I don't, here's what I'd need to find out". That answer is respected here.

## WHEN IT GOES WRONG

# Things that will happen to you

Every row below has bitten this codebase before. Recognising one saves you an afternoon.

| SYMPTOM                                                    | MOST LIKELY CAUSE                                                            | WHAT TO DO                                                                                                                                 |
|------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Meeting stuck in <code>transcribing</code> forever         | Sarvam's callback never arrived                                              | Check <code>sarvam_webhook_triggered_at</code> ; <code>check-recall-status</code> should re-fire it. Look in <code>monitor_events</code> . |
| Transcript is empty but status says <code>completed</code> | The long-audio Sarvam bug, or a chunk came back empty                        | This is a <b>Mission 2 finding</b> . Capture the job id before it ages out.                                                                |
| Speakers show as <code>SPEAKER_00</code>                   | Recall's speaker timeline was empty, so mapping had nothing to match against | <b>Mission 3 finding</b> . Check <code>processing_config</code> on the meeting row.                                                        |
| Whisper crashes on a long file                             | Known OOM above ~15 MB in the edge function                                  | Documented in <code>errors.md</code> as <code>whisper:oom</code> . Don't re-debug it; cite it.                                             |
| Sarvam returns <code>KeyError: 'timestamps'</code>         | Vendor server bug on long audio                                              | Expected. The system auto-falls back to Whisper. Record that it happened.                                                                  |
| <code>git commit</code> rejected                           | The brand pre-commit hook found an off-palette colour or a banned word       | Read the message — it names the file and line. Fix the colour; don't bypass the hook.                                                      |
| Build passes, app breaks at runtime                        | Vite doesn't typecheck                                                       | <code>npx tsc --noEmit</code>                                                                                                              |
| Audio missing from an older meeting                        | <code>prune-recordings</code> clears audio after 30 days                     | By design. Use your own fixtures for anything repeatable.                                                                                  |
| No email arrived from a test                               | Emails are deliberately suppressed for <code>[harness]</code> meetings       | Correct behaviour. Needs <code>HARNESS_EMAILS=true</code> , and only with permission.                                                      |
| Everything broke at once, pipeline-wide                    | Historically: storage bucket full, or an expired API key                     | Check the Supabase storage cap and the secrets first — before debugging any code.                                                          |

## Escalate immediately — don't wait for standup

- Anything touching real customer data
- A production failure you may have caused
- A secret leaked anywhere, even briefly
- Spend heading past the agreed API budget
- A finding that means we're making a false public claim

## How to ask a good question

Four lines, always:

1. **Goal** — what I was trying to do
2. **Attempt** — the exact command or change
3. **Expected vs actual** — with the error pasted verbatim, not paraphrased
4. **My theory** — what I think is happening

Point four is the one that turns a question into a conversation between engineers.

## COST DISCIPLINE

Sarvam, OpenAI and Recall all bill per use. Recall **bot minutes are the expensive one** — prefer fixture files over live bots wherever the test allows it. Keep a running tally of what you've spent in `raw/spend.md` and flag it the moment you're near the agreed ceiling.

## APPENDIX A

# Command cheat sheet

## DAILY

```
# dev server → localhost:8080
npm run dev

# real typecheck (build does NOT do this)
npx tsc --noEmit

# 53 mocked unit tests, <1 s
npm run test:unit

# lint + brand palette check
npm run lint
npm run brand:check
```

## TESTING THE PIPELINE

```
# 11 scenarios against real prod, ~90 s
python3 scripts/pipeline-test/harness.py

# + live Sarvam contract check, ~3 min more
python3 scripts/pipeline-test/harness.py --live

# one scenario only
python3 scripts/pipeline-test/harness.py \
  --only sarvam_webhook_idempotency
```

## OUTPUT QUALITY

```
# 8 evals, 4 deterministic + 4 LLM-judge
python3 scripts/evals/run_evals.py

# pull a real meeting into the eval dataset
python3 scripts/evals/run_evals.py \
  --snapshot <meeting-id>
```

## SARVAM PROBES – YOUR M1 WORKHORSE

```
# production settings
python3 scripts/sarvam_config_test.py <id>

# pin the language, keep translate mode
python3 scripts/sarvam_config_test.py <id> \
  language_code=hi-IN

# native-script output instead of English
python3 scripts/sarvam_config_test.py <id> \
  mode=transcribe language_code=mr-IN

# inspect a job's outputs chunk by chunk
python3 scripts/check_sarvam_job.py <job_id>
```

## AUDIO PREP

```
# match the pipeline's native format exactly
ffmpeg -i in.wav -ar 16000 -ac 1 -b:a 128k out.mp3

# truncate to N seconds (no re-encode)
ffmpeg -i in.mp3 -t 1800 -c copy out-30min.mp3

# exact duration
ffprobe -v error -show_entries format=duration
in.mp3
```

## Useful read-only SQL – run in the Supabase SQL editor

```
-- your test meetings and where they got to
select id, title, status, error_message, sarvam_job_id, created_at
from meetings where title like '[harness]%' order by created_at desc limit 20;

-- transcript size, the fastest signal for silent content loss
select m.id, m.title, length(t.content) as chars,
       jsonb_array_length(t.speaker_segments) as segments
from meetings m join transcripts t on t.meeting_id = m.id
where m.id = '<meeting-id>';

-- what the stuck-meeting monitor has been seeing
select * from monitor_events order by created_at desc limit 30;
```

## ONE LAST THING

# The most useful person in the room is the one holding the numbers.

---

Everything on the EchoBrief landing page is currently a *belief*. By Friday, some of those beliefs will be facts and the rest will be things we have to fix or stop saying. That's the whole value of this week.

You are not here to make the product look good. You're here to find out what's true about it. A report full of failures with numbers attached is a great week's work. A report that says "everything seems fine" means the tests were too gentle.

Be sceptical, be specific, write it down, and ask early.

## ESCALATION

Khush — anything blocking, anything risky, anything customer-facing. Immediately, in the channel.

## BRANCH

`intern/quality-sprint`  
Artefacts in `reports/2026-08-quality-sprint/`

## READOUT

Friday 28 Aug, 16:00.  
Bring the numbers.